

Bases de Datos

Clase 16: Ordenamiento - Selección

Sorting

Los algoritmos de sorting son conocidos en programación

¿Por qué estudiarlos otra vez?

Sorting

Necesitamos ordenar tuplas que exceden por mucho el tamaño de la memoria RAM

External Merge Sort

En los DBMS, se utiliza el algoritmo External Merge Sort

Hablaremos de **Run** como una secuencia de páginas que contiene un conjunto ordenado de tuplas

Algoritmo funciona por **fases**

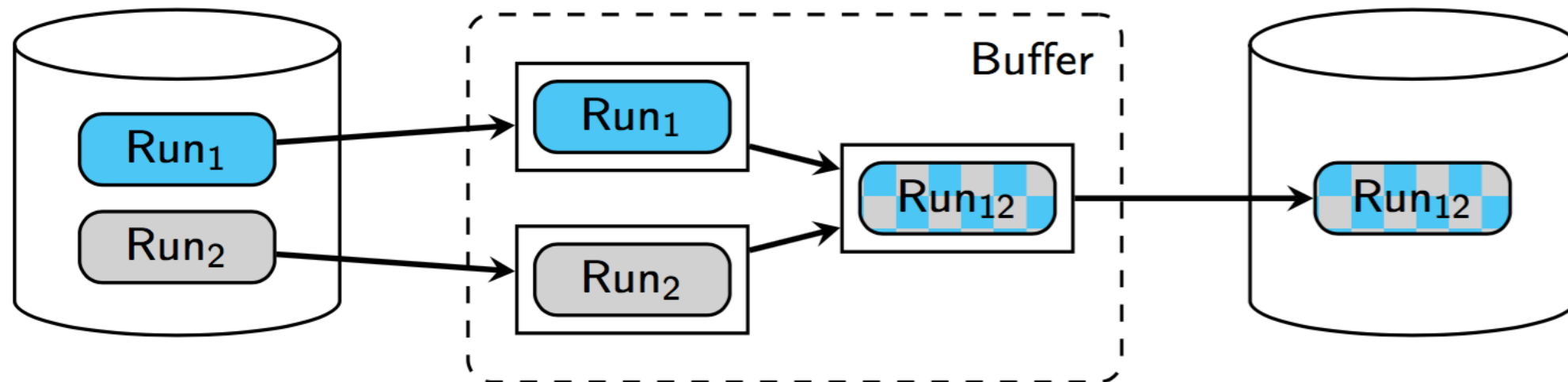
External Merge Sort

Fase 0: creamos los runs iniciales

Fase i:

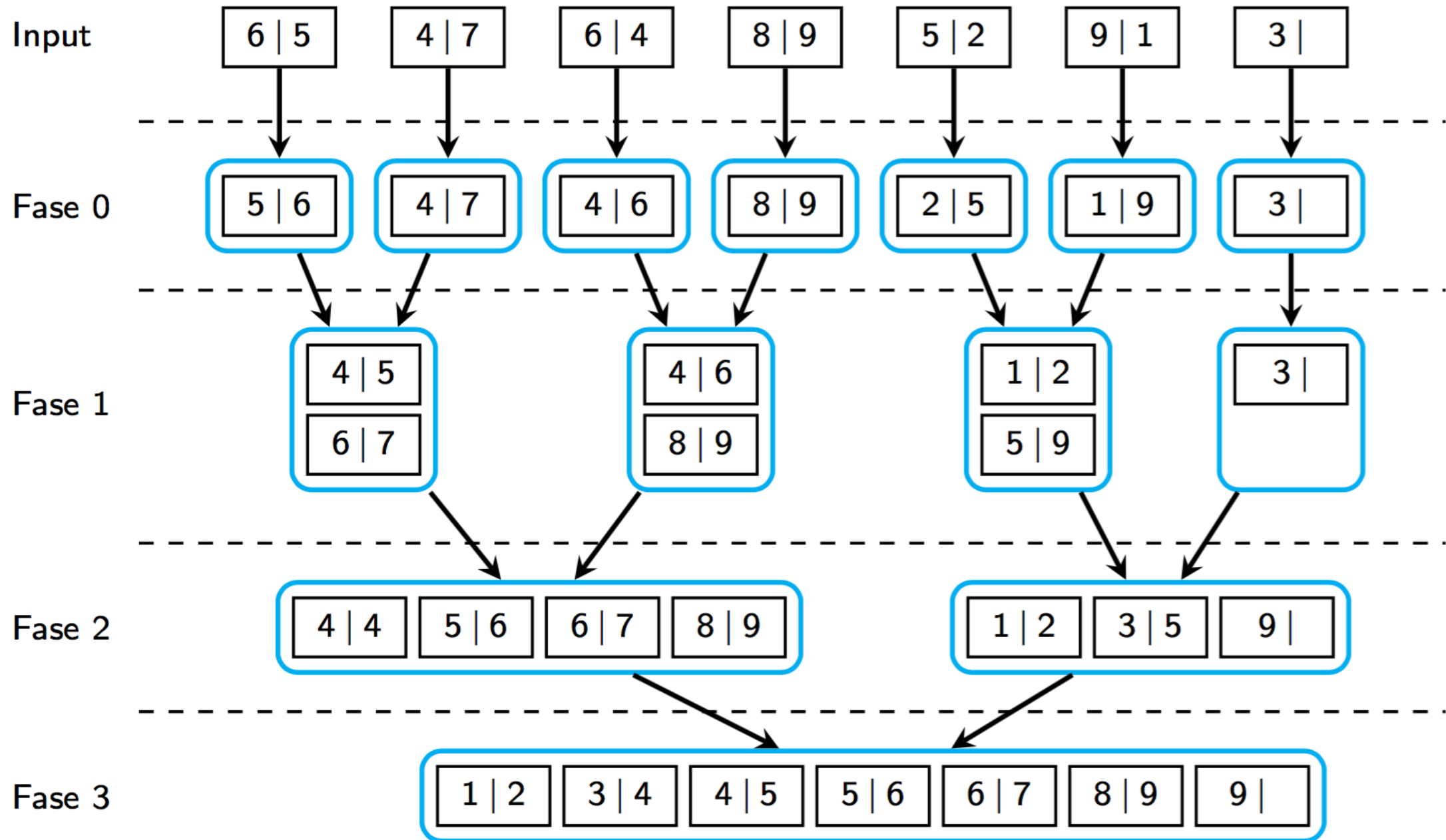
- Traemos los runs a memoria
- Hacemos el merge de cada par de runs
- Almacenamos el nuevo run a disco (i.e. materializamos resultados intermedios)

External Merge Sort



Ojo! cada run se compone de varias páginas, por lo que en cada fase hay un subconjunto de ambos runs en buffer

External Merge Sort



External Merge Sort

Sea **N** número de páginas del archivo

Sabemos que en cada fase se leen todas las páginas y luego se escriben a disco

Número máximo de fases es:

$$\underbrace{1}_{\text{Fase 0}} + \underbrace{\lceil \log_2 N \rceil}_{\text{Fases}}$$

El costo en I/O es:

$$2 \cdot N \cdot (1 + \lceil \log_2 N \rceil)$$

External Merge Sort

Si tenemos una tabla de 8 GB y páginas de 8 KB ~ 1048576 páginas en total

Costo en I/O es:

$$2 \cdot 1048576 \cdot (1 + \lceil \log_2 1048576 \rceil) = 44040192$$

Si cada I/O toma 0.1 ms, ordenar tarda **1.2 horas**

External Merge Sort

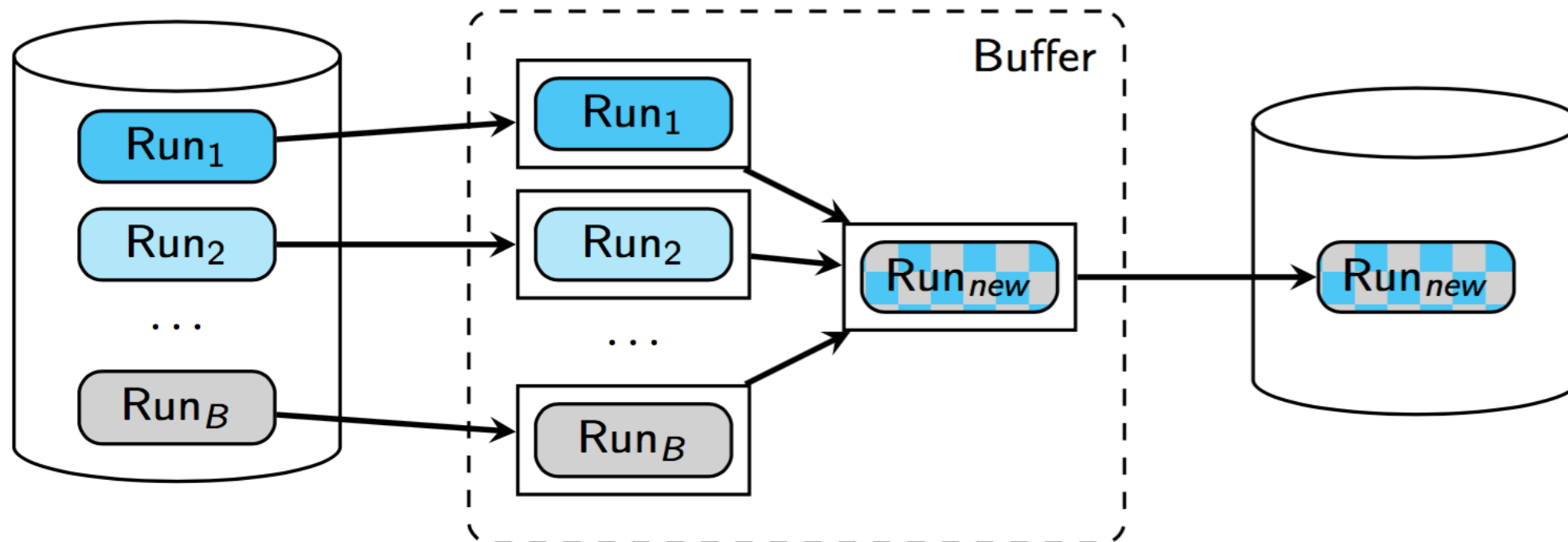
Optimizado

Podemos mejorar el desempeño de nuestro algoritmo

External Merge Sort

Optimizado

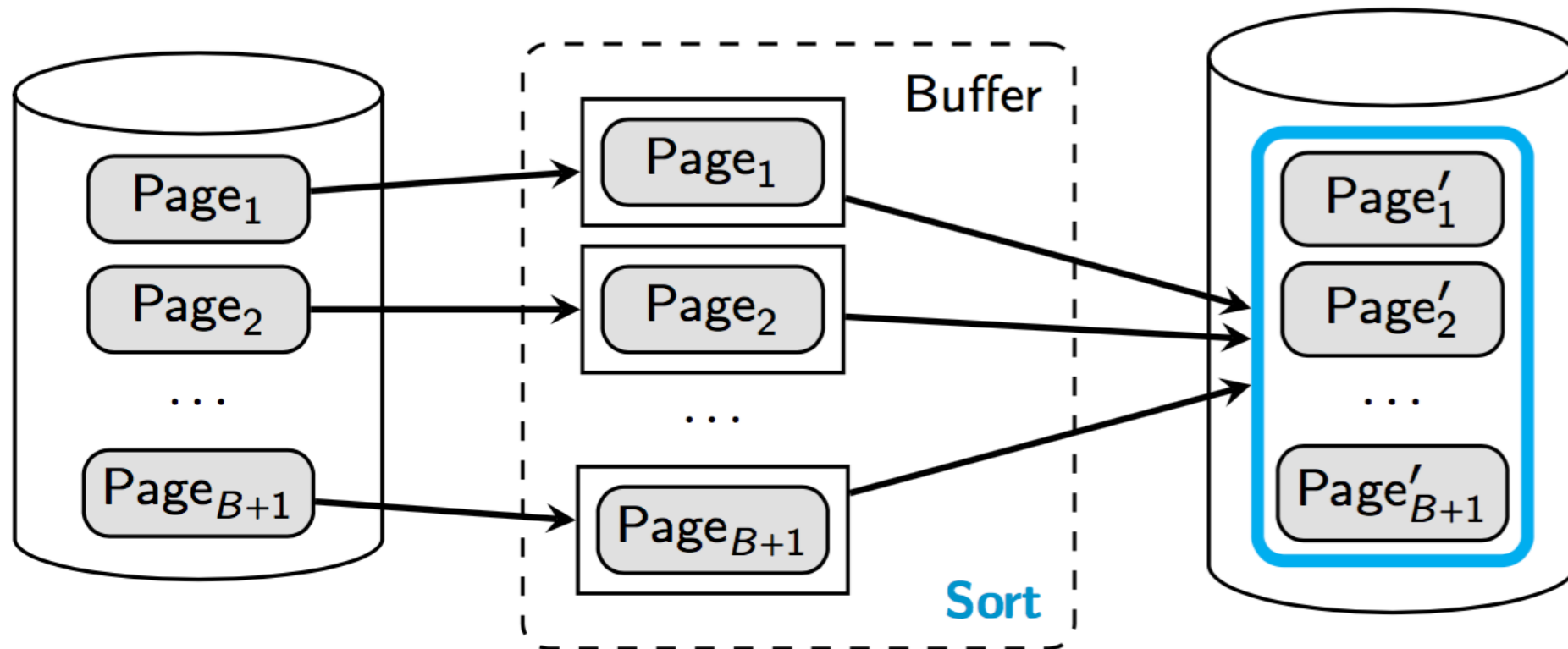
En vez de 3 páginas, tenemos $B + 1$ páginas en buffer



External Merge Sort

Optimizado

Además, tendremos runs iniciales de $B + 1$ páginas



External Merge Sort

Optimizado

Supongamos que queremos ordenar la relación
 $R(a)$ = {20, 19, 18, ..., 1}

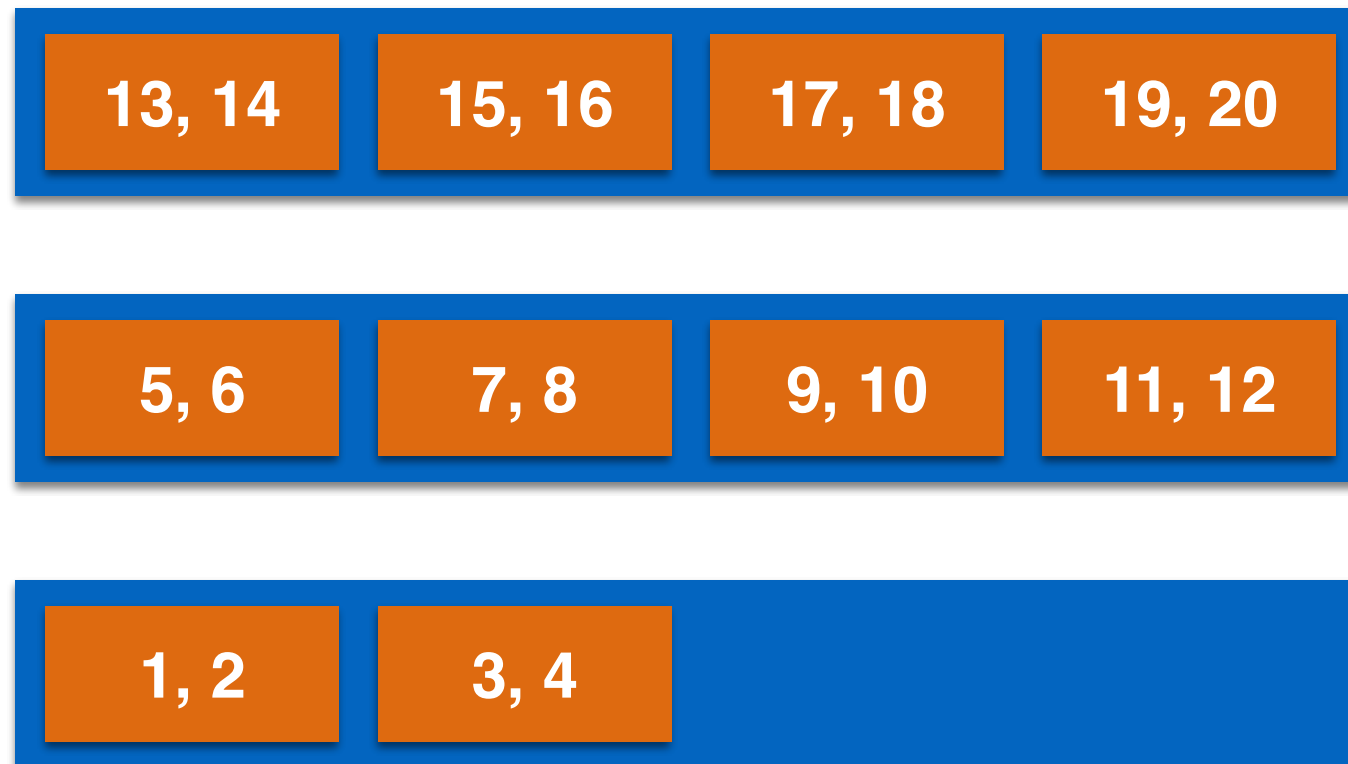
El buffer alcanza para 4 páginas ($B+1 = 4$)

Runs iniciales tienen 4 páginas

External Merge Sort

Optimizado

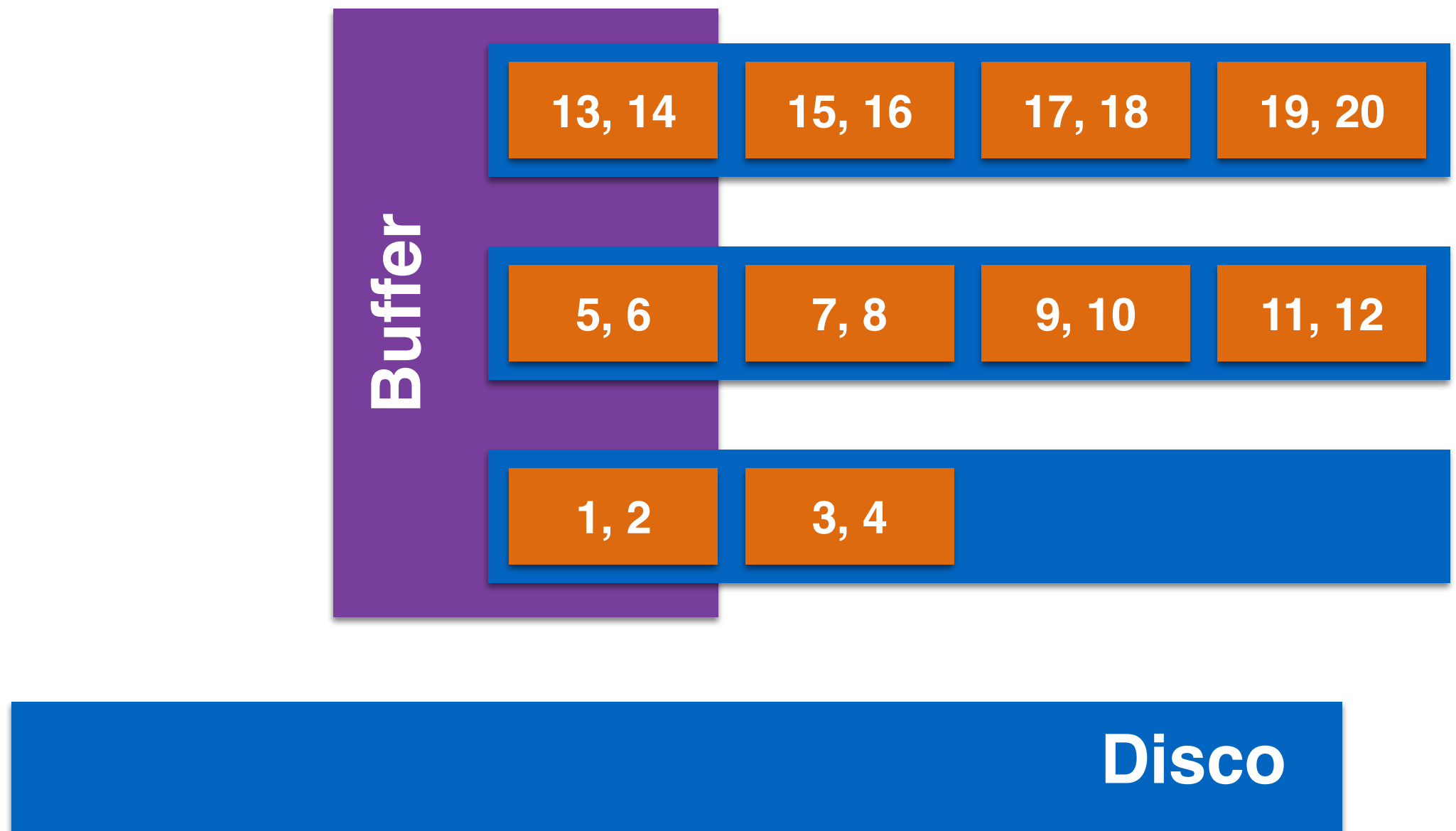
Ordenamos los 3 Runs iniciales:



External Merge Sort

Optimizado

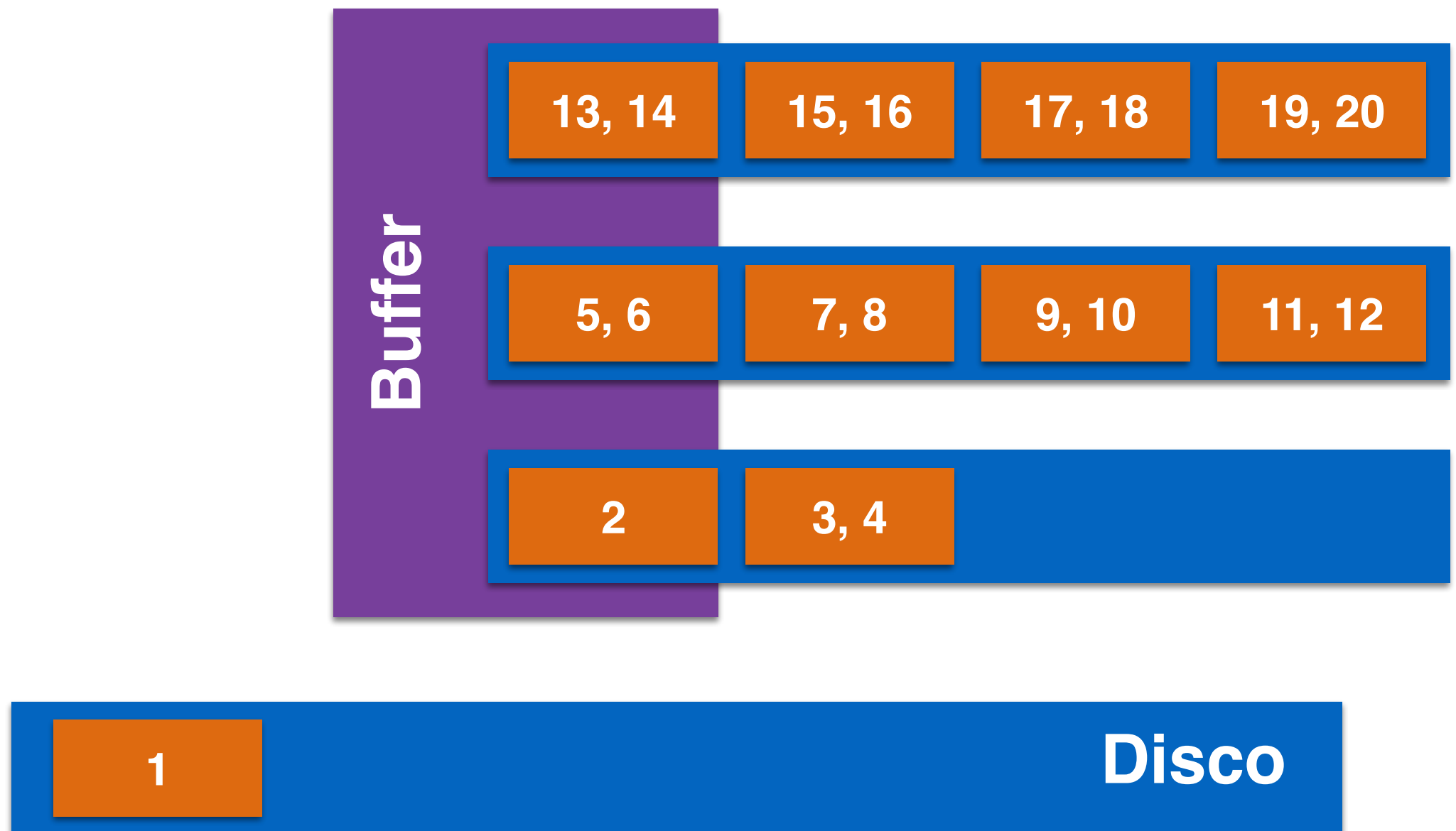
Se van escribiendo las páginas:



External Merge Sort

Optimizado

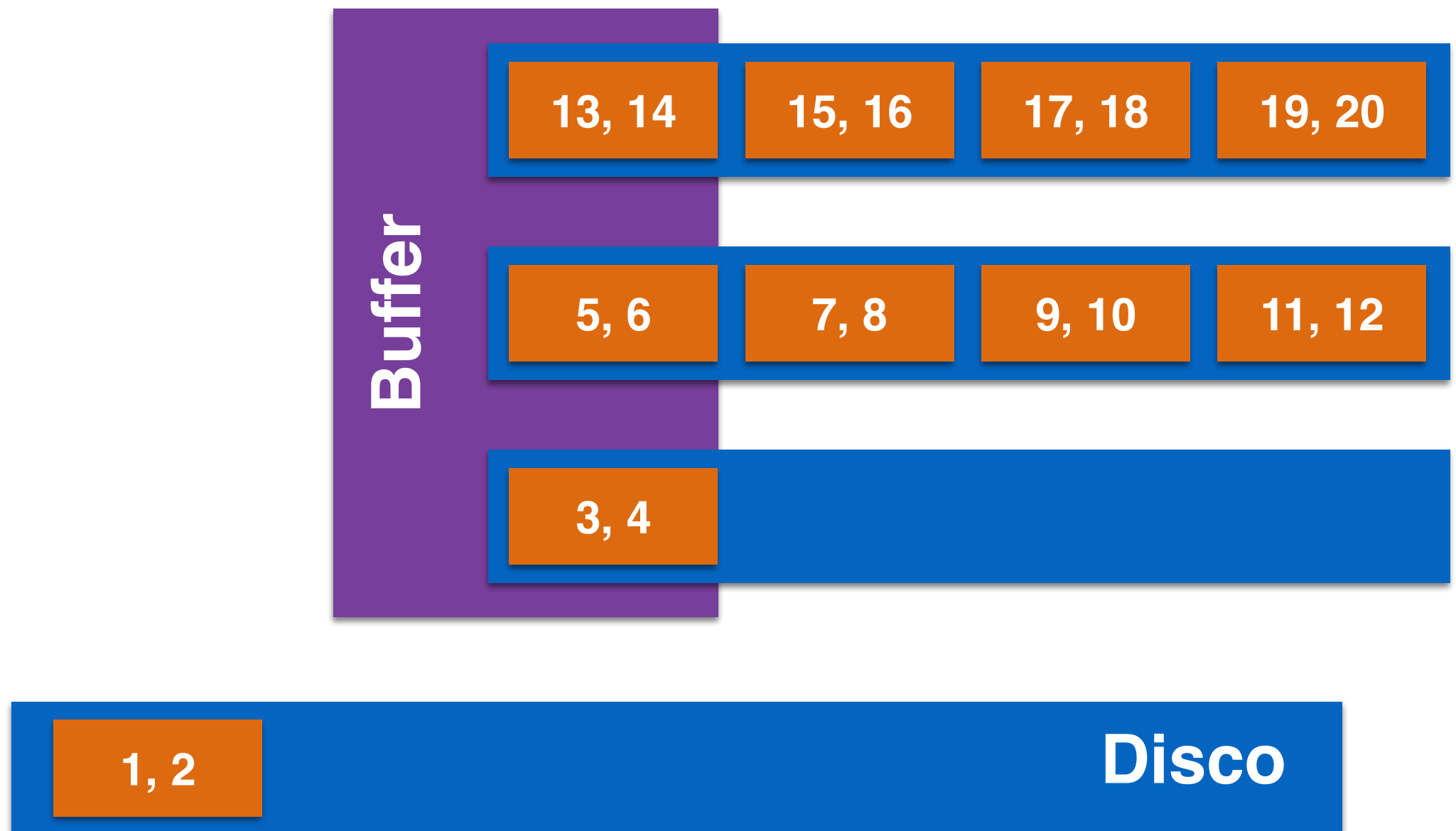
Se van escribiendo las páginas:



External Merge Sort

Optimizado

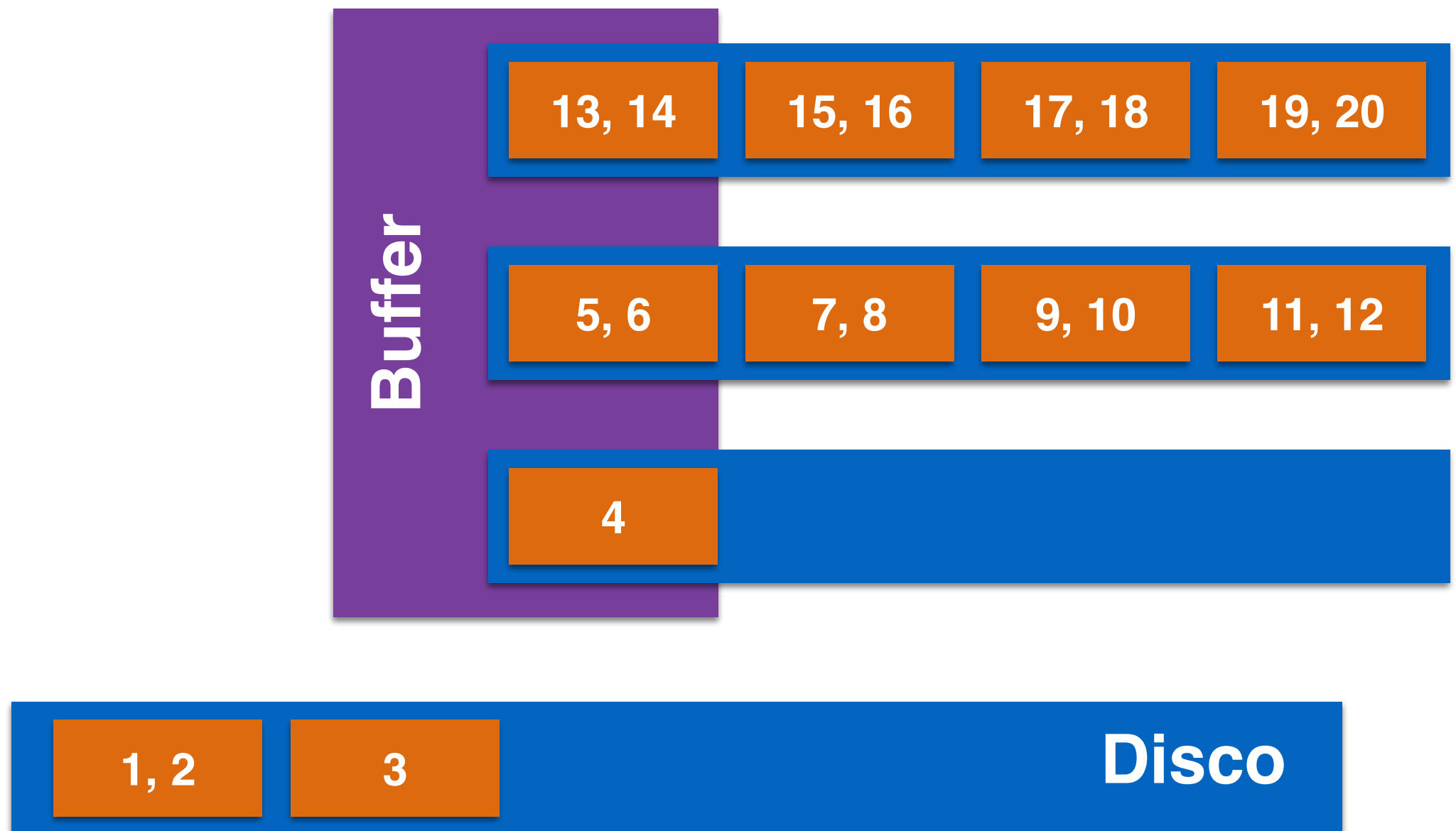
Se van escribiendo las páginas:



External Merge Sort

Optimizado

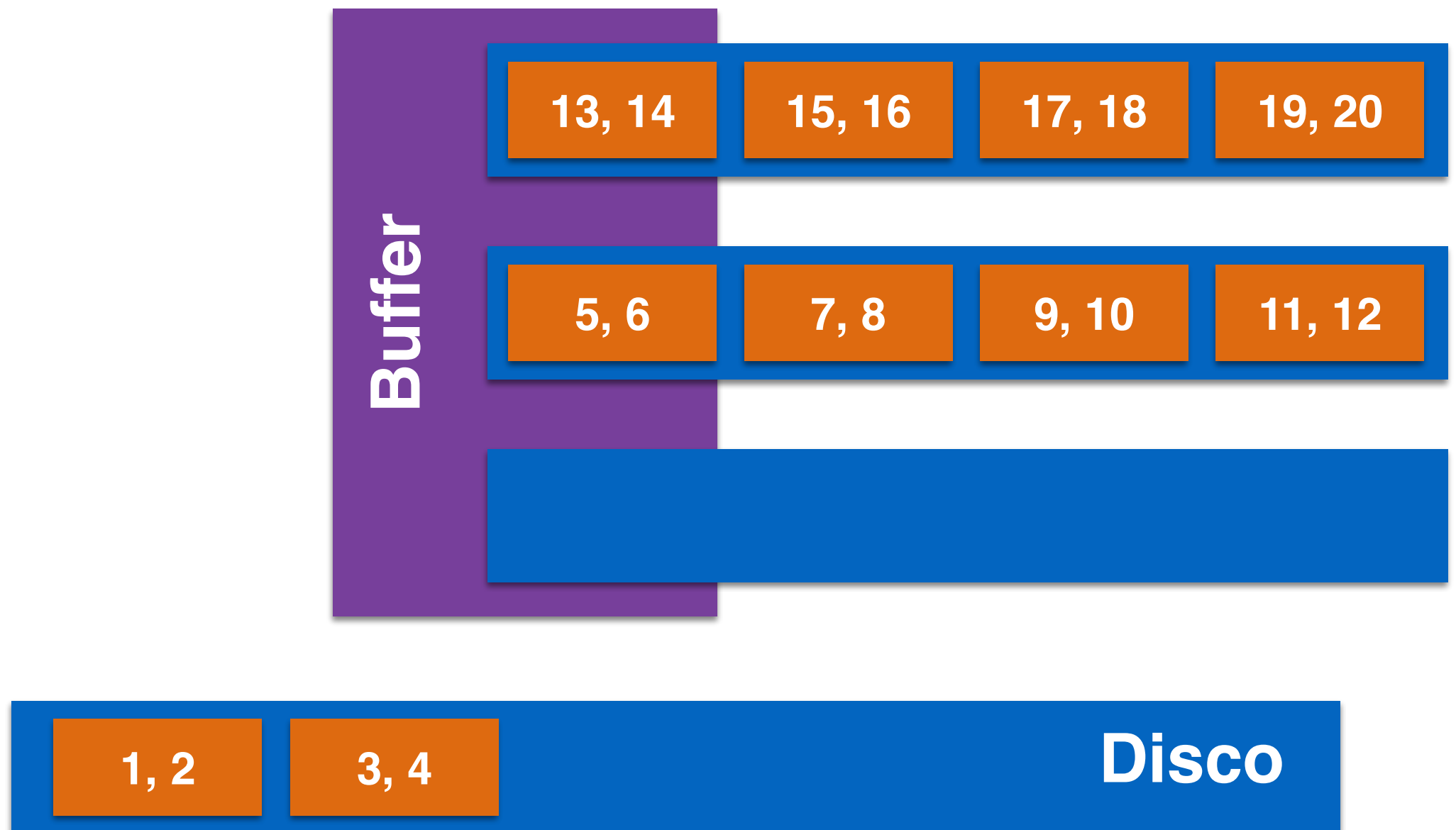
Se van escribiendo las páginas:



External Merge Sort

Optimizado

Se van escribiendo las páginas:



External Merge Sort

Optimizado

Sea **N** número de páginas del archivo y un buffer de tamaño $B + 1$

Número de runs iniciales:

$$\left\lceil \frac{N}{B+1} \right\rceil$$

Número de fases:

$$\underbrace{1}_{\text{Fase 0}} + \underbrace{\left\lceil \log_B \left\lceil \frac{N}{B+1} \right\rceil \right\rceil}_{\text{Fases}}$$

External Merge Sort

Optimizado

Sea **N** número de páginas del archivo y un buffer de tamaño $B + 1$

Costo en I/O:

$$2 \cdot N \cdot \left(1 + \left\lceil \log_B \left\lceil \frac{N}{B+1} \right\rceil \right\rceil\right)$$

External Merge Sort

Optimizado

Si tenemos una tabla de 8 GB y páginas de 8 KB ~
1048576 páginas en total

Memoria RAM para el buffer es de 2 GB, por lo que
 $B + 1 \sim 262145$ páginas

Si cada I/O toma 0.1 ms, ordenar tarda **6.7 minutos**

External Merge Sort

Optimizado

Lo más rápido es ejecutar el algoritmo en 2 fases

$$2 = 1 + \left\lceil \log_B \left\lceil \frac{N}{B+1} \right\rceil \right\rceil$$

↓ despejamos B

$$B \geq \sqrt{N}$$

Si suponemos una tabla de 10^9 páginas (60 TB), sólo necesitamos **240 MB de buffer!**

External Merge Sort

Optimizado

Podemos plantear que si la tabla es de **N** páginas, el costo en I/O de ordenarla es **4N**

¿Qué pasa si no escribimos el último output?
(pipeline)

Selección

El algoritmo de selección cambia dependiendo si es una consulta de igualdad (=) o de rango (<, >)

También depende si el atributo a seleccionar está indexado

Implementa interfaz de iterador lineal

Selección

Sin índice

Si queremos hacer una selección sobre una tabla **R**

```
open()  
  R.open()  
  
next() // retorna el siguiente seleccionado  
  t:= R.next()  
  while t != null do  
    if t satisface condición then  
      return t  
    t:= R.next()  
  return null  
  
close()  
  I.close()
```

Selección

Sin índice

Necesariamente tenemos que recorrer todo **R**

Selección

Con índice y consulta de igualdad

Si queremos hacer una selección sobre una tabla **R** a un atributo indexado con un índice **I**

```
open()  
  I.open()  
  I.search(Atributo = valor)
```

```
next()  
  t := I.next()  
  while t != null do  
    return t  
  return null
```

```
close()  
  R.close()
```

Selección

Con índice y consulta de igualdad

Sólo tenemos que leer las páginas que satisfacen la condición (más I/O si muchas tuplas satisfacen la condición)

Cambia un poco si el índice es Clustered o Unclustered (¿Por qué?)

Si el atributo es llave primaria entonces la operación prácticamente tiene $I/O \sim 1$

Selección

Con índice y consulta de rango

¿Cómo podemos hacer este tipo de consultas de forma eficiente?

Hint Existe un índice especial para hacer esto

Selección

Con índice y consulta de rango

Cuidado - B+ Tree no siempre son la solución

Cuando el rango es una buena porción de la base de datos, y el B+ Tree es unclustered, conviene un Scan Secuencial de toda la Tabla.

Proyección

Algoritmo muy sencillo

```
open()  
  R.open()  
  
next()  
  t := R.next()  
  while t != null do  
    return project(t, atributos)  
  return null  
  
close()  
  R.close()
```


Proyección

Necesariamente tenemos que recorrer todo **R**

Para más detalles tomar
IIC3413 - Implementación de
sistemas de Bases de Datos